GRAPHRAG

Demo: CineGraph - Ứng dụng truy vấn phim được hỗ trợ bởi AI

Nguyen Tri Cao

cao.nguyen@sotatek.com

Ngày 25 tháng 2 năm 2026

Mục lục

Page

1 Giới thiệu **2**

1.1 Bối cảnh & Vấn đề: Điểm hạn chế của RAG truyền thống với câu hỏi toàn cục (Global Sensemaking) 2

2 Tổng quan **3**

2.1 RAG Baseline: Vector Search + Chunking 3

2.2 GraphRAG 3

3 POC **12**

3.1 Thiết lập Môi trường & Dữ liệu 12

3.2 End-to-End 14

3.3 Evaluation 15

4 Kết quả & Phân tích **18**

4.1 Kết quả thực nghiệm 18

4.2 Phân tích kết quả 19

5 Khuyến nghị & Kết luận **20**

5.1 Khuyến nghị 20

5.2 Kết luận 21

References **21**

1 Giới thiệu

1.1 Bối cảnh & Vấn đề: Điểm hạn chế của RAG truyền thống với câu hỏi toàn cục (Global Sensemaking)

1.1.1 Bối cảnh:

Nhắc lại một chút về RAG - một kỹ thuật để cải thiện đầu ra của các mô hình ngôn ngữ lớn (LLM) bằng cách sử dụng thông tin từ tài liệu cụ thể để nó có thể trả lời tốt trong phạm vi cụ thể đó. Kỹ thuật này là một phần quan trọng của hầu hết các công cụ dựa trên LLM và phần lớn các phương pháp RAG sử dụng similarity vector làm kỹ thuật tìm kiếm (search engine), hướng tiếp cận này được gọi là Baseline RAG.

1.1.2 Vấn đề

Các kỹ thuật RAG đã cho thấy triển vọng trong việc giúp các LLM lý luận về các tập dữ liệu riêng tư - dữ liệu mà LLM không được đào tạo và chưa bao giờ thấy trước đó, chẳng hạn như nghiên cứu độc quyền, tài liệu kinh doanh hoặc giao tiếp của một doanh nghiệp. Baseline RAG được tạo ra để giúp giải quyết vấn đề này, nhưng khi triển khai sẽ có những tình huống mà Baseline RAG hoạt động rất kém:

- Baseline RAG gặp khó khăn khi nối kết các điểm thông tin. Điều này xảy ra khi trả lời một câu hỏi đòi hỏi phải đi qua các mảnh thông tin khác nhau, đôi khi thông tin sẽ khá rắc rối, sau đó được tổng hợp lại. Và đôi khi do sự phức tạp hoặc dư thừa từ dữ liệu đưa vào mà LLM trả lời sai.
- Baseline RAG hoạt động rất kém (chứ không muốn nói là fail) khi được yêu cầu hiểu một cách toàn diện một thông tin nào đó trên các bộ sưu tập dữ liệu lớn hoặc thậm chí trên các tài liệu lớn. Ví dụ đưa cho nó một tập lớn các điều lệ công ty và bảo nó tóm tắt tất cả điều lệ công ty, thì baseline RAG xịt keo ngay.

Để giải quyết vấn đề này, GraphRAG ra đời - 1 phương pháp mới của Microsoft Research. GraphRAG, sử dụng các LLM để tạo ra một đồ thị tri thức dựa trên một tập hợp đầu vào. Hiểu đơn giản thì thay vì lưu trữ các chunk text và embedding nó để search, thì GraphRAG biểu diễn và lưu trữ dưới dạng đồ thị, cùng các kỹ thuật nâng cao về gom nhóm, tìm kiếm để đưa ra câu trả lời tối ưu nhất, khắc phục được 2 nhược điểm bên trên. GraphRAG cho thấy sự cải thiện đáng kể, vượt trội hơn so với các phương pháp khác đã được áp dụng trước đó đối với các tập dữ liệu riêng tư.

2 Tổng quan

2.1 RAG Baseline: Vector Search + Chunking

RAG (Retrieval-Augmented Generation), thường được gọi là Vector RAG hoặc Naive RAG, là nền tảng phổ biến nhất hiện nay. Nó hoạt động dựa trên một quy trình tuần tự đơn giản: chia nhỏ văn bản, biểu diễn thành vector, tìm kiếm theo ngữ nghĩa và tổng hợp câu trả lời. Luồng xử lý được mô tả ở hình 2.1 .

2.1.1 Chunking

Mục đích các tài liệu dài thành các đoạn văn bản nhỏ (chunks) có kích thước phù hợp để nhúng (embed) và nạp vào context window của LLM.
Phương pháp phổ biến:

- Chunk theo kích thước cố định: 512 hoặc 1024 tokens, có thể có phần chồng lấn (overlap) giữa các chunk để tránh mất ngữ cảnh.
- Chunk theo ngữ nghĩa: Dựa trên cấu trúc văn bản (đoạn văn, tiêu đề).

Hạn chế: Việc chia cắt có thể làm đứt gãy mạch ngữ nghĩa tự nhiên và các mối quan hệ logic vốn trải dài qua nhiều đoạn văn.

2.1.2 Vector Search

- **Embedding Model:** Mô hình (như text-embedding-3-small) chuyển đổi mỗi chunk văn bản thành một vector số học đa chiều, mã hóa ý nghĩa ngữ nghĩa của nó.
- **Vector Database:** Cơ sở dữ liệu chuyên dụng (như LanceDB, Pinecone, Weaviate) lưu trữ các vector này cùng với siêu dữ liệu (metadata) tham chiếu về chunk gốc. Nó hỗ trợ tìm kiếm nhanh chóng các vector "gần nhau"nhất trong không gian đa chiều.

2.2 GraphRAG

2.2.1 Index

Quá trình Index của GraphRAG với mục tiêu lưu chung là lưu trữ thông tin để phục vụ Query, và việc lưu trữ này dưới dạng đồ thị. Trong đồ thị thì có 2 thành phần không thể thiếu, đó là Node và Edge.

1)TextUnit

TextUnit là một mảnh (chunk) văn bản được tạo ra từ Document để làm “đơn vị phân tích” cho toàn bộ pipeline. Kích thước chunk (tính theo token), overlap, và cách bám theo ranh giới dữ liệu (document boundary / group-by) đều cấu hình được. TextUnits đóng vai trò rất quan trọng cho truy vết nguồn (breadcrumbs/provenance): các tri thức trích xuất (entity/relationship/claim) có thể “tham chiếu ngược” về TextUnits để biết nó đến từ đoạn nào.

2)Extract entities, Relationships, Claims

Đây là bước LLM "đọc hiểu"và trừu tượng hóa thông tin từ TextUnits thành cấu trúc tri thức.

- **Entities:** Có title, type, description. Các đối tượng có tên và ý nghĩa (Người, Tổ chức, Địa điểm, Khái niệm...). Ví dụ: "Công ty SotaTek", "Ronaldo".
- **Relationships:** Có source, target, description. Mối liên kết có ý nghĩa giữa hai thực thể. Ví dụ: "Quantum Systems sở hữu NeoChip".
- **Claims (Optional):** là workflow độc lập để trích xuất claims từ TextUnits. Claims là các phát biểu thực tế (positive factual statements) có trạng thái đánh giá và ràng buộc thời gian. Docs mô tả claims được lưu dưới dạng covariates và thường cần prompt tuning để phù hợp ngữ cảnh dữ liệu. Các phát biểu sự thật hoặc sự kiện cụ thể liên quan đến thực thể. Ví dụ: "Cổ phiếu của NeoChip tăng mạnh trong tuần đầu giao dịch."

Merge + Summarize: Các subgraph theo TextUnit sẽ được gộp (entity trùng title-type được merge; relationship trùng source-target được merge), rồi dùng LLM để tóm tắt danh sách mô tả thành một mô tả gọn cho mỗi node/edge trong graph.

3)Community detection (hierarchical clustering/Leiden)

Sau khi có graph Entities–Relationships, GraphRAG chạy Hierarchical Leiden community detection để tạo cấu trúc phân cụm phân cấp (hierarchy) và tìm các cụm node liên quan chặt chẽ .

Mục đích: Từ đồ thị thực thể-quan hệ (các node và edge), tìm ra các cụm (community) gồm những thực thể có kết nối chặt chẽ với nhau hơn là với phần còn lại của đồ thị. Thuật toán Leiden: Được GraphRAG sử dụng vì nó tạo ra các community liên thông tốt và có thể chạy đệ quy để phát hiện các community con bên trong một community lớn.

- Cấp 0 (Root/Gốc): 1 community lớn chứa toàn bộ đồ thị.
- Cấp 1: 5 community chính (ví dụ: "Chủ đề Công nghệ", "Chủ đề Chính sách").
- Cấp 2: 15 community con (ví dụ: "AI Ethics"bên trong "Chủ đề Công nghệ").

4)Generate Community Summaries

Đây là bước tổng hợp tri thức quan trọng nhất, tạo ra "chỉ mục thông minh"cho GraphRAG. Khi đã có hierarchy communities, GraphRAG tạo Community Reports bằng LLM để cung cấp hiểu biết tổng quan ở nhiều mức:

- Community top-level → report mang tính “toàn cục” của cả graph
- Community level thấp → report mang tính “cục bộ” của một cụm nhỏ

Generate Community Reports: LLM tạo report cho từng community để nêu thông tin “distinct” của community, và report sẽ tham chiếu các key entities, relationships, và claims trong cấu trúc community đó. Summarize Community Reports: Sau đó mỗi report lại được tóm tắt thành bản ngắn (summary) để dùng nhanh khi truy vấn. Đồ thị cùng các communities có thể xem hình 2.2 minh họa dưới đây:

2.2.2 Query

1)Global Search

Kỹ thuật này phù hợp với các câu hỏi mang tính toàn diện, tổng quan về dữ liệu, truy vấn toàn cục để "Thấy Rừng", những câu hỏi mang tính khái quát chung mà RAG truyền thống có khả năng fail cao. Nhờ vào việc gom nhóm các entities và relationships vào các community, thì lúc này ta sẽ dựa vào mô tả của các communities đó. Được xử lý theo cơ chế Map-Reduce

- **Map:**
  - Các community summaries được shuffle và chia thành các chunks với fix token size
  - Dùng LLM sinh ra mỗi câu trả lời cho 1 chunk cùng với score (0-100) (tùy bài toán)
- **Reduce:**
  - Sort các câu trả lời theo score
  - Đưa vào LLM các câu trả lời này cho tới khi limi token để sinh ra global answer (target)

Câu hỏi phù hợp: Câu hỏi cần hiểu biết tổng thể, phân tích xu hướng, tóm tắt chủ đề trên toàn bộ dữ liệu.

Ví dụ: "Các chủ đề công nghệ nổi bật nhất trong năm là gì?", "Đâu là những rủi ro chính được đề cập xuyên suốt các báo cáo dự án?"

2)Local Search

Kỹ thuật này phù hợp với các câu hỏi sâu về một chủ đề nào đó, phù hợp để suy luận sâu hơn về các thực thể và các mối quan hệ láng giềng, truy vấn cục bộ để "Xem Cây"

- **Xác định thực thể liên quan:** Local Search xác định một tập hợp các thực thể từ knowledge graph có liên quan về mặt ngữ nghĩa với query của người dùng. Các thực thể này đóng vai trò là endpoint vào knowledge graph, cho phép trích xuất thêm các chi tiết liên quan như các entities được kết nối/liên quan, relationships, biến cố của thực thể và summary reports.
- **Trích xuất thông tin:** Ngoài ra, Local Search cũng trích xuất các đoạn văn bản có liên quan từ các tài liệu đầu vào thô được liên kết với các entities đã xác định.
- **Ưu tiên và lọc:** Các nguồn dữ liệu ứng viên này sau đó được ưu tiên và lọc để phù hợp với một cửa sổ ngữ cảnh đơn lẻ có kích thước được xác định trước, được sử dụng để tạo phản hồi cho truy vấn của người dùng.

Câu hỏi phù hợp: Câu hỏi xoáy sâu vào một đối tượng cụ thể, yêu cầu phân tích chi tiết, mối quan hệ và ngữ cảnh xung quanh.

Ví dụ: "Công ty Alpha đã hợp tác với những ai và trong các dự án nào?", "Nguyên nhân và diễn biến chính của sự cố X là gì?"

3)DRIFT Search

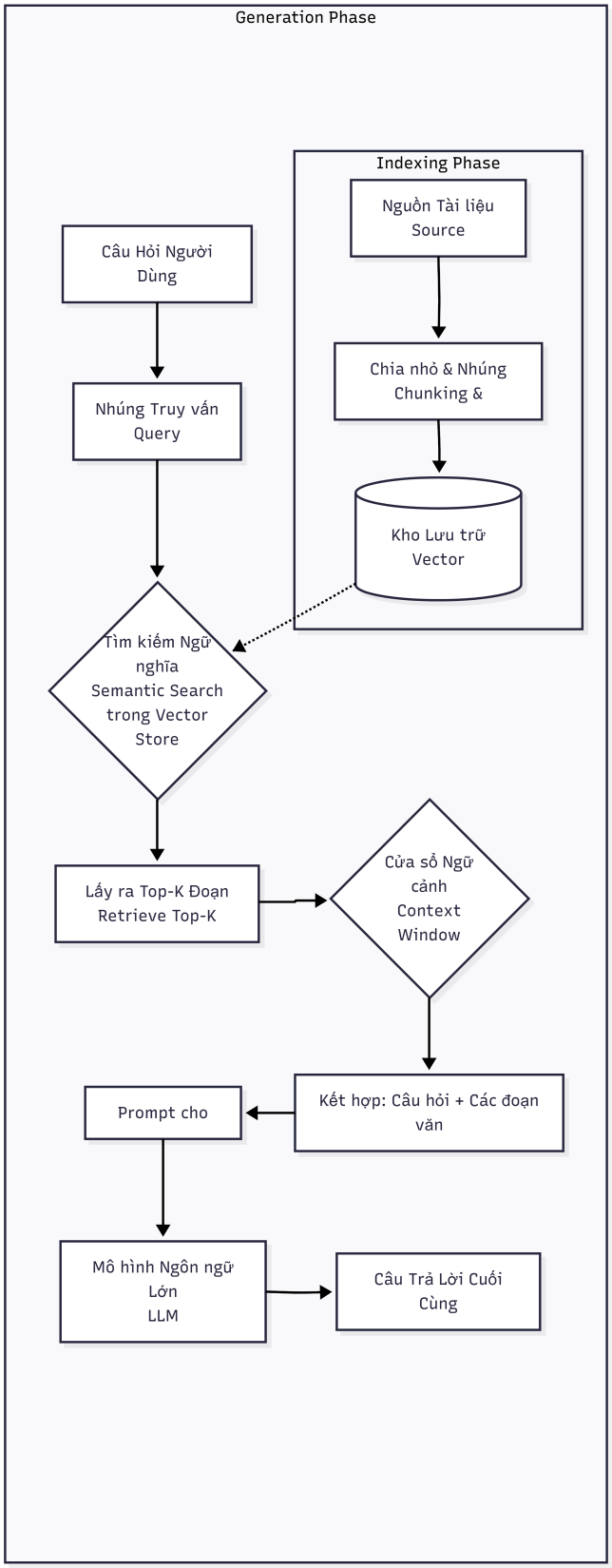
Là một phương pháp lai (hybrid) kết hợp điểm mạnh của cả Local và Global Search. "DRIFT"(tạm hiểu: lan tỏa) mở rộng phạm vi tìm kiếm cục bộ bằng cách đưa thêm ngữ cảnh toàn cục từ các Community Reports có liên quan, truy vấn Kết hợp để "Xem Cả Rừng lẫn Cây"

- Bắt đầu như Local Search: Xác định thực thể/chủ đề trọng tâm và thu thập ngữ cảnh đồ thị cục bộ.
- Mở rộng bằng Global Context: Hệ thống xác định các Community Reports nào có chứa hoặc liên quan đến thực thể/chủ đề đó.
- Kết hợp ngữ cảnh: Ngữ cảnh cục bộ (chi tiết, sâu) được bổ sung bởi các báo cáo cộng đồng liên quan (tổng quan, rộng). Sự kết hợp này cung cấp cả chiều sâu và bề rộng.

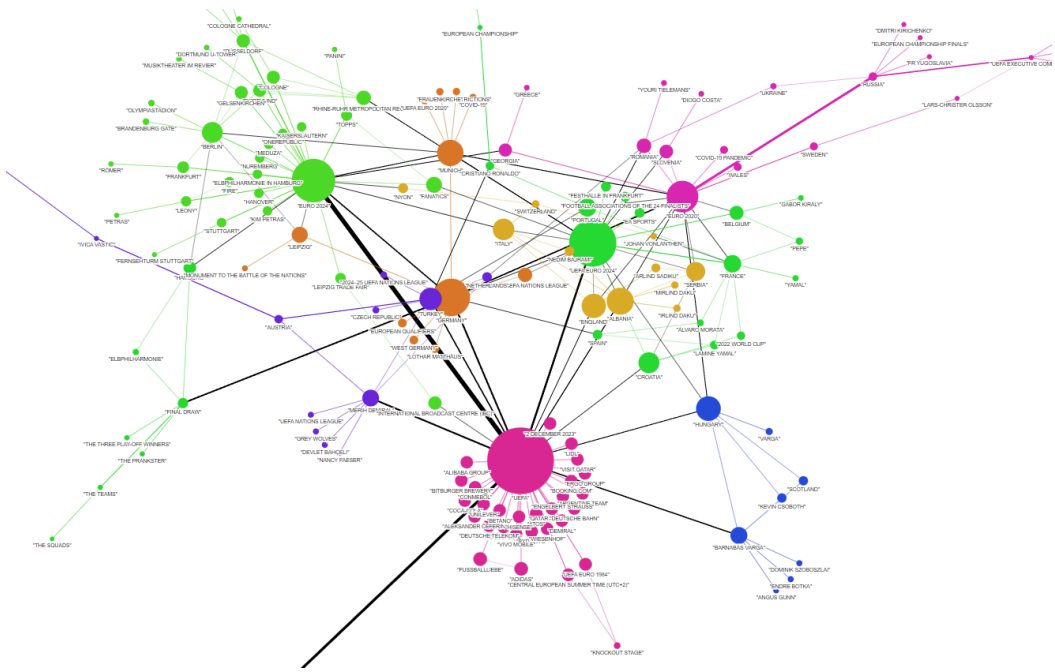
- Tạo câu trả lời nâng cao: LLM sử dụng ngữ cảnh kết hợp phong phú này để tạo ra câu trả lời vừa chi tiết, vừa đặt đối tượng vào bức tranh tổng thể lớn hơn.

Câu hỏi phù hợp: Câu hỏi phức tạp, đa lớp, yêu cầu hiểu sâu một đối tượng NHƯNG đồng thời cần đánh giá tác động, vị trí hoặc ý nghĩa của nó trong toàn cục.

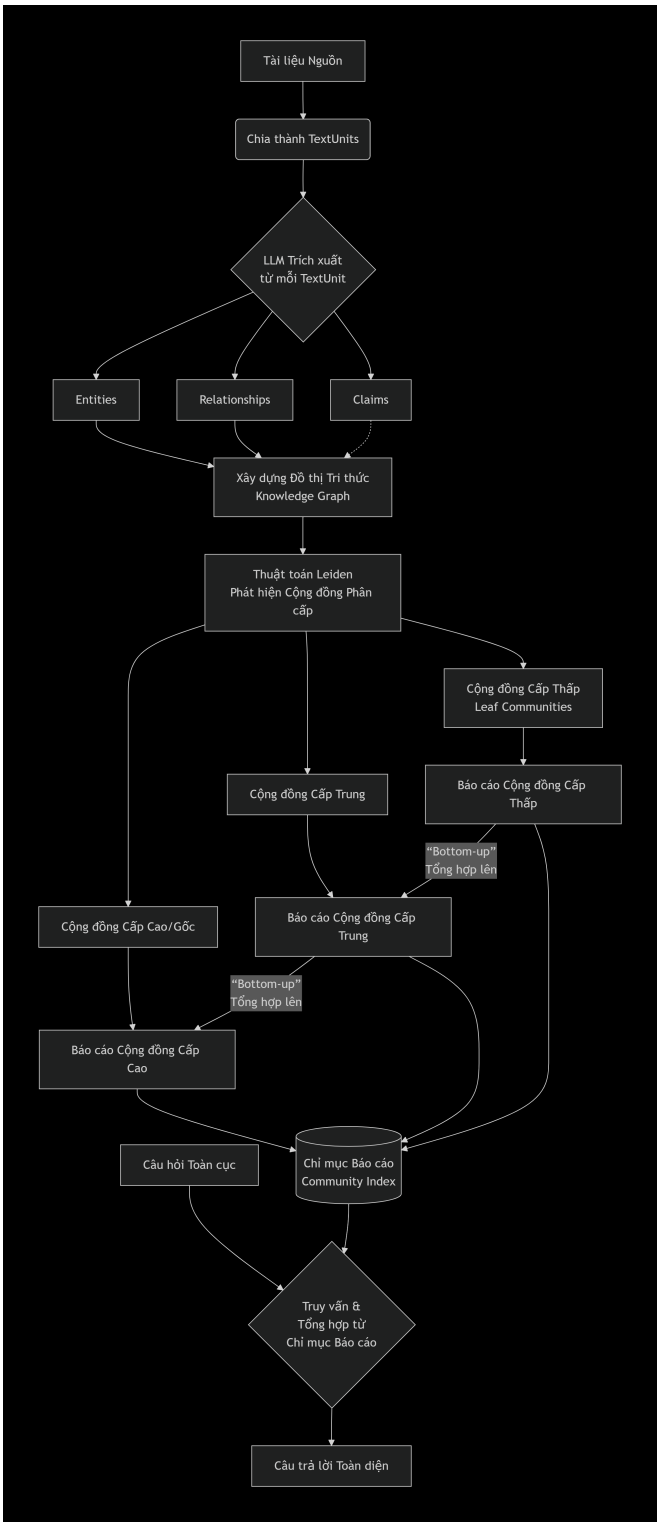
Ví dụ: "Vụ bê bối của công ty Beta ảnh hưởng thế nào đến toàn ngành và các chủ đề thảo luận công cộng?", "Phát minh X không chỉ là gì, mà nó còn thúc đẩy những xu hướng nghiên cứu lớn nào khác?"



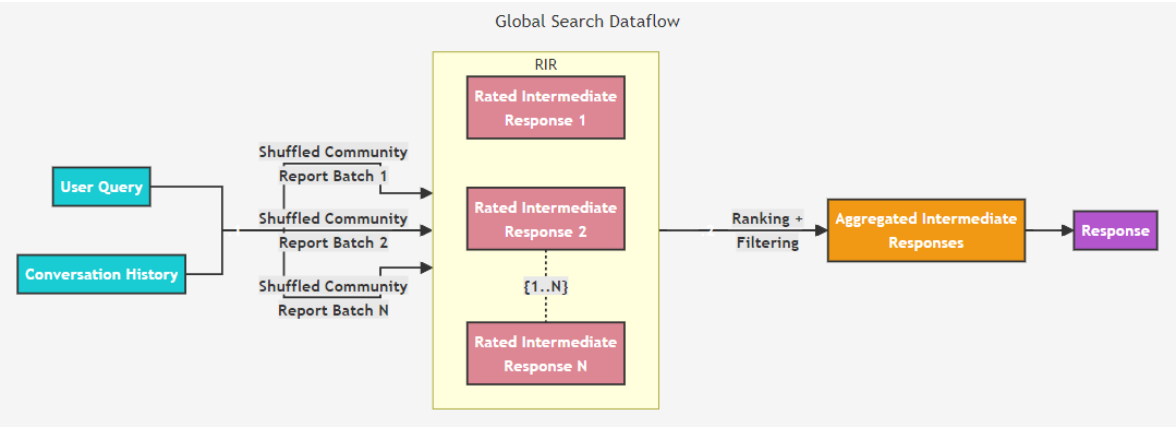
Hình 2.1: RAG Pipeline



Hình 2.2: Node, Relationship



Hình 2.3: Pipeline of GraphRAG



Hình 2.4: Global Search

3 POC

3.1 Thiết lập Môi trường & Dữ liệu

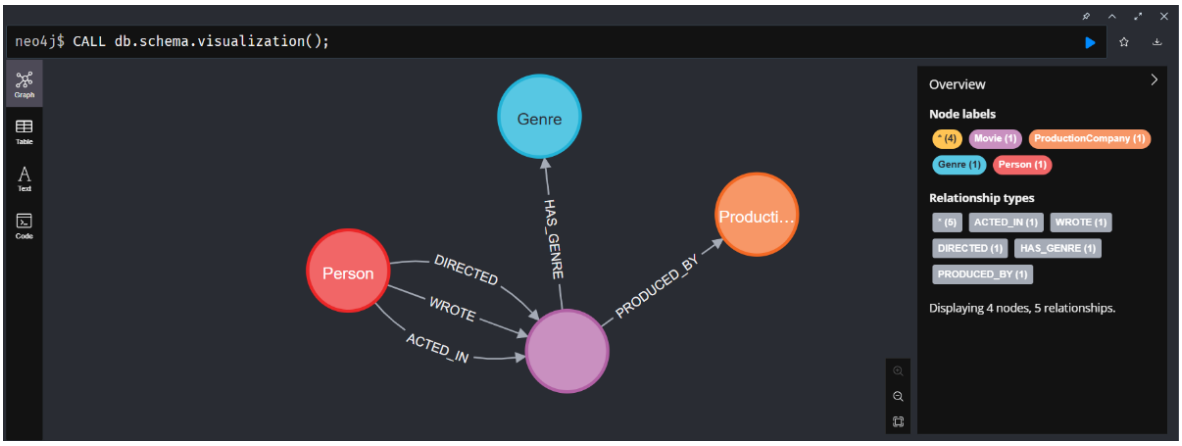
POC được triển khai theo kiến trúc 3 lớp (Neo4j – Backend API – Frontend) chạy bằng Docker Compose nhằm đảm bảo khả năng tái lập (reproducibility) và giảm sai lệch môi trường giữa các máy triển khai. Cấu hình mặc định sử dụng các cổng: Neo4j HTTP `7474`, Neo4j Bolt `7687`, Backend `8000`, Frontend `3000`. Backend khởi động với cơ chế *lifespan* để kết nối Neo4j và tự động import dữ liệu CSV nếu đồ thị rỗng.

3.1.1 Corpus

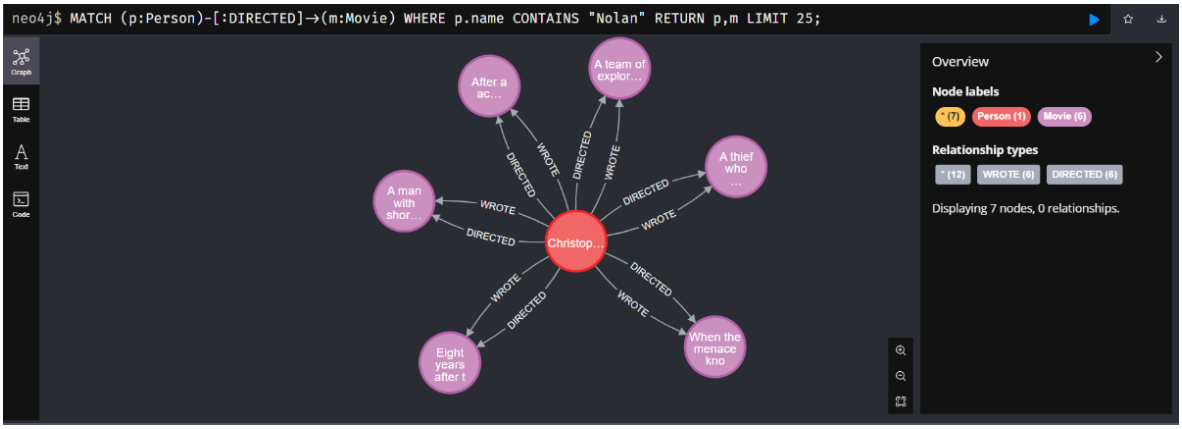
Trong POC này, corpus được hiểu là tập dữ liệu nguồn phục vụ truy hồi và sinh câu trả lời. Dữ liệu đầu vào ở dạng CSV (ví dụ: `backend/movies.csv`) gồm các trường mô tả phim (tựa đề, năm, thời lượng, mô tả, điểm đánh giá, số lượt bình chọn) và các trường thực thể/quan hệ (đạo diễn, biên kịch, diễn viên, thể loại, hãng sản xuất).ó thể thay thế bằng tập hợp tài liệu nội bộ (X tài liệu) nơi thực thể như "khách hàng – hợp đồng – điều khoản – sản phẩm" có quan hệ rõ.

Dữ liệu được nạp vào Neo4j theo mô hình thực thể–quan hệ (entity–relationship):

- **Node**:
  - `Movie`: `imdb_title_id`, `original_title`, `year`, `duration`, `description`, `avg_vote`, `votes`
  - `Person`: `name`
  - `Genre`: `name`
  - `ProductionCompany`: `name`
- **Relationship**:
  - `(:Person)-[:ACTED_IN]->(:Movie)`
  - `(:Person)-[:DIRECTED]->(:Movie)`
  - `(:Person)-[:WROTE]->(:Movie)`
  - `(:Movie)-[:HAS_GENRE]->(:Genre)`
  - `(:Movie)-[:PRODUCED_BY]->(:ProductionCompany)`



Hình 3.1: Schema trực quan



Hình 3.2: Ví dụ về mối liên hệ node trong neo4j

Quá trình import sử dụng `MERGE` để đảm bảo tính *idempotent* (chạy lại không tạo trùng dữ liệu), đồng thời tạo index cho các thuộc tính truy vấn thường gặp (ví dụ: `Movie.original_title`, `Person.name`, `Genre.name`) để giảm độ trễ truy vấn trong giai đoạn demo.

3.1.2 GraphRAG

GraphRAG trong POC được triển khai theo hướng *graph-native retrieval* trên Neo4j, tập trung vào việc truy hồi đúng theo quan hệ thay vì chỉ dựa trên độ tương tự ngữ nghĩa.

Pipeline khái quát:

1. **Phân tích truy vấn (Query analysis)**: LLM phân loại câu hỏi và trích xuất thực thể/điều kiện (ví dụ: `actor`, `director`, `genre`, hoặc `keywords`). (‘llm_client.analyze_query‘) Kết quả trả về là một JSON có cấu trúc để điều phối bước truy hồi.

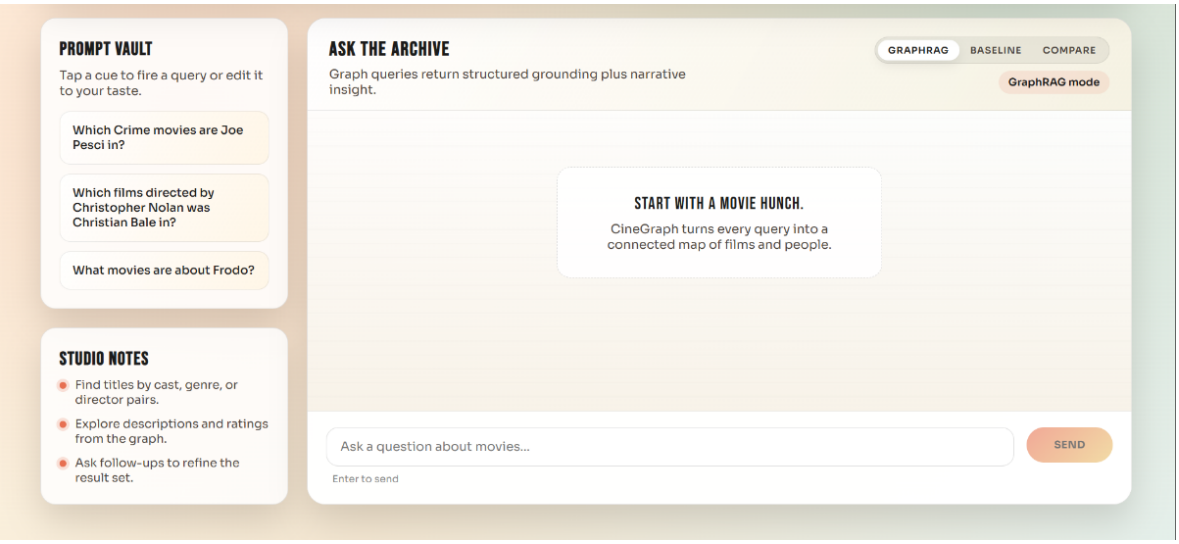
2. **Truy hồi theo đồ thị (Graph retrieval)**: hệ thống ánh xạ loại truy vấn sang các mẫu Cypher tương ứng: (‘graphrag.execute_graphrag_query‘)

- Actor–Genre: `(Person)-[:ACTED_IN]->(Movie)-[:HAS_GENRE]->(Genre)`
- Director–Actor: `(Person)-[:DIRECTED]->(Movie)<-[:ACTED_IN]-(Person)`
- Keyword search (khi không suy ra quan hệ): lọc theo `Movie.description` (và/hoặc title) bằng biểu thức phù hợp.

3. **Sinh câu trả lời (Grounded generation)**: LLM nhận vào câu hỏi và danh sách kết quả truy hồi (grounding set) để sinh phần bình luận/tóm tắt (‘llm_client.generate_response‘). Trong POC, output được chuẩn hóa theo Markdown gồm: *Commentary*, *Matching Filmography*, và *Performance Stats*.

Điểm nhấn của GraphRAG trong POC là khả năng **ràng buộc truy hồi theo quan hệ** nhằm giảm nhiễu và hạn chế lỗi suy diễn quan hệ (ví dụ: trả về phim có đạo diễn đúng nhưng diễn viên sai, hoặc ngược lại). Đây là tiền đề để mở rộng lên GraphRAG đầy đủ theo hướng community detection và hierarchical summarization khi dữ liệu lớn và câu hỏi thiên về tổng hợp toàn cục.

3.2 End-to-End



Hình 3.3: Giao diện

Luồng End-to-End của hệ thống được thiết kế theo hướng tối giản cho demo và dễ quan sát chất lượng:

1. **Khởi chạy hệ thống**: Docker Compose dựng 3 service (Neo4j, Backend, Frontend). Backend chờ Neo4j sẵn sàng (healthcheck) và thực hiện import CSV khi đồ thị chưa có dữ liệu.

2. **Tương tác người dùng**: Người dùng nhập câu hỏi trong Frontend. UI hỗ trợ 3 chế độ:

- **GraphRAG**: gọi `/api/chat`
- **Baseline**: gọi `/api/baseline`
- **Compare**: gọi `/api/compare` và hiển thị song song kết quả Baseline vs GraphRAG

3. **Xử lý tại Backend**:
   - Với GraphRAG: (analysis → graph retrieval → generation)
   - Với Baseline: (keyword retrieval → generation)
   - Với Compare: chạy cả hai pipeline, trả về kết quả và metrics theo cấu trúc thống nhất
4. **Quan sát/giải thích**: Output trả về dạng Markdown giúp kiểm tra nhanh:
   - **Grounding**: bảng kết quả truy hồi (phim/metadata)
   - **Answer**: phần bình luận LLM
   - **Metrics**: thời gian phân tích, thời gian truy hồi, thời gian sinh, tokens/second, v.v.

3.3 Evaluation

3.3.1 Bộ câu hỏi và phân nhóm

Tệp `eval_questions.jsonl` gồm **100 câu hỏi**, chia thành:

- **40 câu local**: truy vấn trực tiếp theo thực thể/quan hệ (actor, director, genre, hoặc keyword cụ thể).
- **60 câu global/complex**: yêu cầu tổng hợp, nhiều điều kiện hoặc suy luận đa bước (multi-hop).

3.3.2 Thiết lập đánh giá và thu thập dữ liệu

Batch evaluation được chạy bằng script `backend/eval/run_eval.py` thông qua các endpoint:

- `POST /api/eval/baseline`
- `POST /api/eval/graphrag`
- `POST /api/eval/compare`

Mỗi pipeline trả về:

- `response_text`: câu trả lời dạng text (dùng cho RAGAS).
- `contexts`: danh sách context strings (dùng cho RAGAS, context precision/recall).

- `retrieved`: danh sách record JSON (phục vụ phân tích truy hồi).

Output được lưu vào `eval_outputs.jsonl` (theo từng câu hỏi) và `eval_summary.json` (thống kê).

3.3.3 Chỉ số đánh giá

Hiệu năng: đo latency end-to-end của endpoint `/api/eval/compare`.
Truy hồi: thống kê kích thước tập truy hồi (`len(retrieved)`) theo pipeline và theo nhóm local/global.
Chất lượng (mục tiêu): chạy RAGAS để tính Faithfulness, Answer Relevancy, Context Relevancy, Context Precision/Recall. Việc chạy RAGAS yêu cầu cấu hình đầy đủ provider (LLM/embeddings) và dependency; trong lần chạy đầu, phần RAGAS chưa kích hoạt thành công (được phân tích ở Chương 4).

Ghi chú: Để đảm bảo nội dung mới hiển thị rõ ràng trong tài liệu, phần mô tả đánh giá cũ (bị lỗi mã hóa) được ẩn đi bằng `\iffalse...\fi`. Đánh giá trong POC được tổ chức theo hai lớp: **định lượng** (metrics) và **định tính** (case study).

3.3.4 Phân tích định tính (4 ví dụ)

Các ví dụ dưới đây trình bày theo dạng minh họa . Khi chạy demo thực tế có thể xuất log/answer từ hệ thống và thay thế bằng kết quả đo đạc thực tế.

Ví dụ 1 (Local – truy vấn trực tiếp). **Câu hỏi:** "Phim thể loại Crime có Joe Pesci tham gia là những phim nào?"

Baseline RAG : Trả về danh sách phim chứa từ khóa "Crime" trong mô tả; đôi khi lẫn các phim không có Joe Pesci nếu phần mô tả nhắc đến "crime" nhiều.

GraphRAG : Truy vấn theo quan hệ `(:Person {name:'Joe Pesci'})-[:ACTED_IN]->(:Movie)-[:HAS_GE` `{name:'Crime'})`, do đó danh sách thường "sạch" hơn và ít nhiễu.

Nhận xét: GraphRAG thường vượt trội ở *Context Relevance* và giảm lỗi sai quan hệ (relationship errors).

Ví dụ 2 (Global/Multi-hop – điều kiện kép director+actor). **Câu hỏi:** "Những phim do Christopher Nolan đạo diễn mà Christian Bale đóng là phim nào?"

Baseline RAG: Có thể thu hồi các đoạn mô tả chứa Nolan hoặc Bale, nhưng việc lấy giao (*intersection*) hai điều kiện dễ sai nếu không có đủ bằng chứng đồng thời trong cùng một chunk/ngữ cảnh.

GraphRAG: Truy vấn quan hệ `(:Person {director})-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(:Pers` `{actor})` nên đảm bảo thỏa điều kiện kép một cách tường minh.

Nhận xét: GraphRAG thường cải thiện *Faithfulness* và *Answer Relevance* cho các truy vấn dựa trên quan hệ.

Ví dụ 3 (Aggregate/Summarization – tổng hợp chủ đề). **Câu hỏi:** "Các chủ đề/nội dung nổi bật trong các phim liên quan tới Frodo là gì?"

Baseline RAG: Dễ tập trung vào một vài chunk có từ "Frodo", dẫn đến câu trả lời thiên lệch và thiếu bao phủ.

GraphRAG: Nếu đồ thị có liên kết nhân vật/diễn viên/series, GraphRAG có thể gom đúng tập phim liên quan trước, sau đó LLM tổng hợp dựa trên dữ liệu đã được *grounding*; kết quả thường bao phủ hơn.

Nhận xét: GraphRAG thường cải thiện *Comprehensiveness* và *Diversity* khi cần "tổng hợp trên đúng tập dữ liệu".

Ví dụ 4 (Trường hợp Baseline có thể tốt hơn). **Câu hỏi:** "Tóm tắt mô tả phim X" (câu hỏi rất cục bộ).

Baseline RAG: Chỉ cần truy hồi đúng record/chunk của phim X; trả lời nhanh và chi phí thấp.

GraphRAG: Bước phân tích câu hỏi và/hoặc truy vấn đồ thị có thể là dư thừa (tăng token/latency), trong khi lợi ích từ cấu trúc quan hệ không đáng kể.

Nhận xét: Baseline phù hợp khi câu hỏi đơn giản, truy hồi "một điểm" và không cần suy luận nhiều bước.

4 Kết quả & Phân tích

4.1 Kết quả thực nghiệm

4.1.1 Hiệu năng end-to-end

Độ trễ khi chạy batch 100 câu hỏi (endpoint `/api/eval/compare`) được thống kê từ `eval_summary.json` như sau:

- Mean: **0.508 s**
- p50: **0.435 s**
- p95: **1.172 s**
- Max: **1.448 s**

Các số liệu này phù hợp cho POC/demo. Để tối ưu cho sản xuất, cần tách latency theo từng pha (query analysis, retrieval, generation).

4.1.2 Đặc tính truy hồi (kích thước grounding set)

Từ `eval_outputs.jsonl`, kích thước tập truy hồi được thống kê như Bảng 4.1.

Pipeline	Mean	p50	p95	Max
Baseline	4.68	4	16	19
GraphRAG	64.39	81	109	110

Bảng 4.1: Thống kê kích thước tập truy hồi trên 100 câu hỏi

Phân tách theo loại câu hỏi:

- **Local (40 câu)**:
  - Baseline: mean 3.5, p50 2, p95 7, max 9
  - GraphRAG: mean 19.75, p50 14, p95 68, max 92
- **Global (60 câu)**:
  - Baseline: mean 5.47, p50 4, p95 16, max 19
  - GraphRAG: mean 94.15, p50 98, p95 109, max 110

4.2 Phân tích kết quả

4.2.1 Nguyên nhân GraphRAG truy hồi quá rộng trong lần chạy batch

Trong cấu hình hiện tại, GraphRAG phụ thuộc vào bước *query analysis* để chọn đúng mẫu truy vấn Cypher (actor–genre, director–actor, ...). Khi lớp LLM không sẵn sàng hoặc không được cấu hình đầy đủ, hệ thống rơi về chế độ fallback tách từ theo khoảng trắng, làm tập từ khóa chứa nhiều từ chung (ví dụ: *which, films, directed, was, in*). Kết hợp với nhánh keyword trong GraphRAG không giới hạn `LIMIT`, truy hồi trở nên rất rộng và tạo grounding set lớn (đặc biệt cho nhóm global).

Ngược lại, baseline có bước loại stopwords đơn giản nên số lượng truy hồi nhỏ hơn và ổn định hơn. Điều này cho thấy để GraphRAG phát huy lợi thế (truy hồi theo quan hệ), cần ổn định lớp phân tích câu hỏi hoặc bổ sung parser dự phòng.

4.2.2 Tác động tới chất lượng câu trả lời và RAGAS

Trong lần chạy thực nghiệm, phần RAGAS chưa chạy thành công do thiếu dependency provider (log báo thiếu module `anthropic`). Ngoài ra, để RAGAS cho kết quả có ý nghĩa, cần đảm bảo pipeline trả lời thực sự sử dụng LLM (không rơi vào fallback). Vì vậy, giai đoạn tiếp theo cần:

- Cài đặt và cấu hình đầy đủ môi trường RAGAS (dependencies + provider env vars).
- Chạy lại batch để thu thập các metric: Faithfulness, Answer Relevancy, Context Relevancy, Context Precision/Recall.
- Bổ sung một phần ground-truth (tối thiểu 20–30 câu) để đánh giá recall một cách chặt chẽ.

5 Khuyến nghị & Kết luận

5.1 Khuyến nghị

5.1.1 Khi nào nên dùng GraphRAG

GraphRAG phù hợp khi doanh nghiệp cần:

- Hiểu toàn cục trên dữ liệu lớn: tổng hợp xu hướng, insight, hoặc trả lời câu hỏi cấp quản trị.
- Suy luận đa bước: câu hỏi yêu cầu nối chuỗi quan hệ giữa nhiều thực thể.
- Quan hệ thực thể quan trọng: "ai–làm gì–ở đâu–khi nào", "điều khoản–áp dụng–đối tượng", "sản phẩm–phiên bản–rủi ro–biện pháp".
- Giảm hallucination dạng quan hệ: câu trả lời cần bảo đảm "đúng cặp" (đúng người–đúng dự án–đúng điều khoản).

5.1.2 Khi nào Baseline RAG vẫn phù hợp

Baseline RAG (đặc biệt vector RAG) vẫn là lựa chọn tối ưu khi:

- Dữ liệu chủ yếu unstructured đơn giản, câu hỏi mang tính tra cứu trực tiếp.
- Cần time-to-value nhanh, triển khai gọn nhẹ, ít phụ thuộc vào quy trình entity extraction/graph modeling.
- Hệ thống cần chi phí thấp, latency thấp, và chấp nhận một mức sai số nhất định trong các truy vấn quan hệ phức tạp.

5.1.3 Chi phí scalability

So sánh tổng quan:

- Baseline vector RAG:
  - Index nhanh (embedding batch). vars để RAGAS chạy ổn định.
  - Query rẻ hơn (không cần LLM analyze nếu có router đơn giản).

  - Tuy nhiên chất lượng suy luận quan hệ phụ thuộc vào dữ liệu có “nằm cùng chunk” hay không.

- GraphRAG:
  - Tốn công xây dựng graph (entity extraction, chuẩn hóa thực thể, mapping).
  - Có thể tốn LLM tokens khi indexing (nếu trích xuất entity/summarization bằng LLM).
  - Query có thể đắt hơn do phân tích câu hỏi + truy vấn đồ thị + tổng hợp.
  - Đổi lại, chất lượng cho global/multi-hop thường ổn định hơn và kiểm soát được theo cấu trúc.

5.2 Kết luận

GraphRAG đem lại lợi ích kinh doanh rõ nhất trong các bài toán cần tổng hợp toàn cục, multi-hop reasoning, và đảm bảo đúng quan hệ thực thể, từ đó giảm rủi ro trả lời sai trong các ngữ cảnh nhạy cảm (pháp lý, tuân thủ, báo cáo điều hành). Baseline RAG vẫn là lựa chọn hiệu quả khi nhu cầu chủ yếu là tra cứu cục bộ và tối ưu time-to-market.